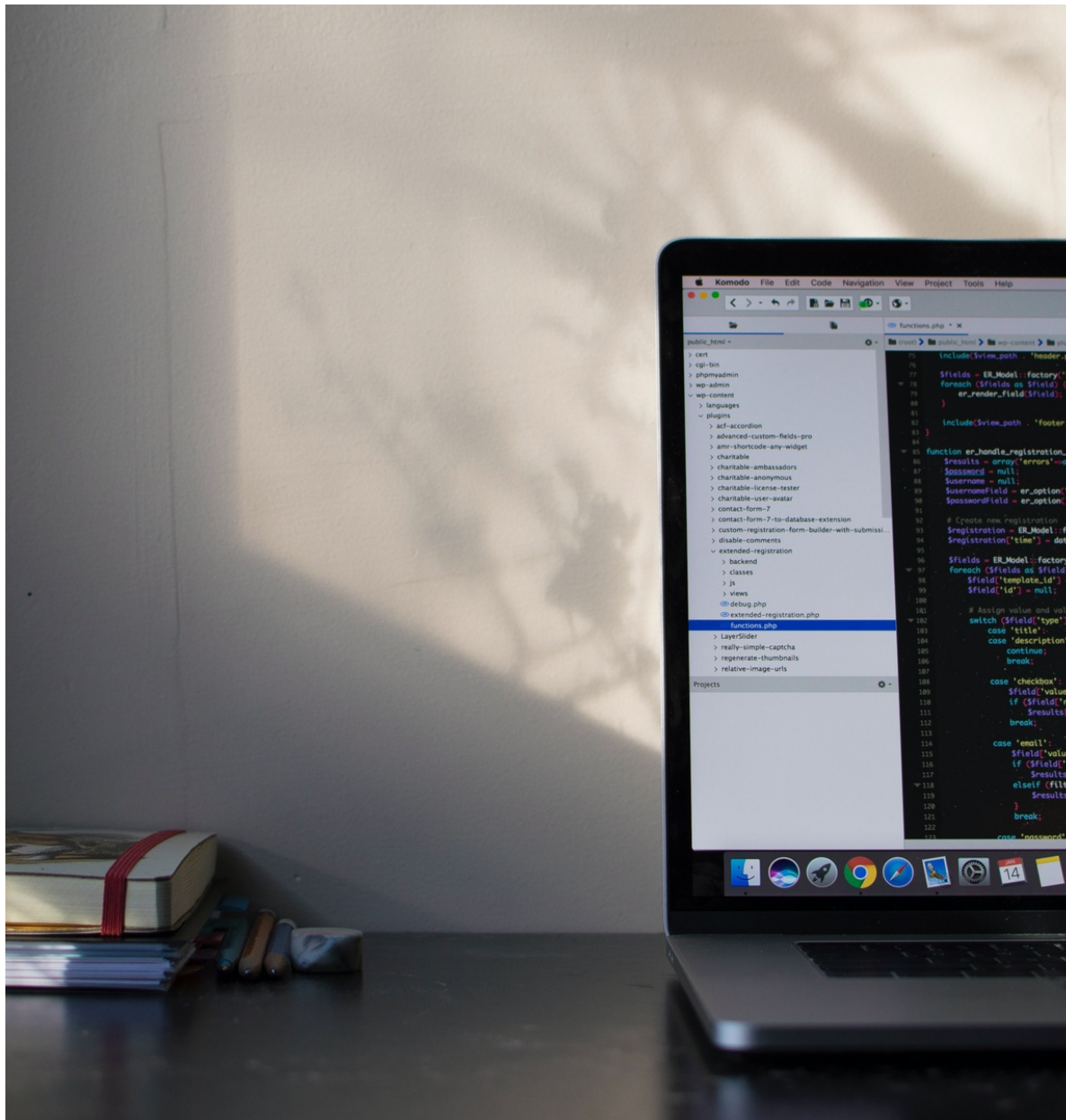


```
37         if path:
38             self.file = open(os.path.join(path, "requirements.txt"), "w")
39             self.file.seek(0)
40             self.fingerprints.update(e.requirements)
41
42     @classmethod
43     def from_settings(cls, settings):
44         debug = settings.getbool("SUPERFILTER_DEBUG")
45         return cls(job_dir(settings), debug)
46
47     def request_seen(self, request):
48         fp = self.request_fingerprint(request)
49         if fp in self.fingerprints:
50             return True
51         self.fingerprints.add(fp)
52         self.file:
```

Object Oriented Programming Presentation

An intro to classes, objects, inheritance, encapsulation, abstraction and polymorphism in programming.



What is Object Oriented Programming?

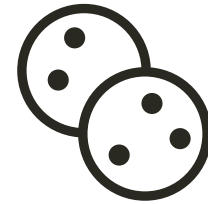
Object Oriented Programming (OOP) is a programming paradigm that uses classes and objects to model real-world concepts. OOP relies on key principles like encapsulation, inheritance, abstraction, and polymorphism to organize and process complex programs.

Building Blocks of OOP



Classes

The blueprint that defines the structure and behavior.

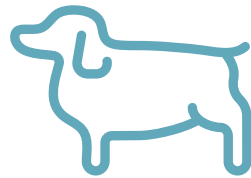


Objects

An individual instance created from that class.

In OOP, you design classes to represent real-world things, and then you create objects to interact with those things within your program. It's a way to organize code that makes it more understandable, reusable, and maintainable.

Objects in OOP



Attributes

Attributes are like the characteristics or properties of an object. They define the state or quality of that object.



Methods

Methods are like the actions or behaviors that an object can perform. They define what the object can do.

Attributes define what the object is like, and methods define what the object can do. It's like creating a virtual version of something, complete with its appearance and abilities!

Four Principles of OOP



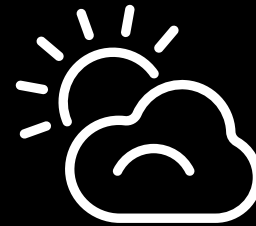
Inheritance

Child classes inherit and extend attributes and behaviors from parent classes



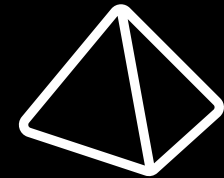
Encapsulation

Combining data and functions into a single component and hiding unnecessary details



Abstraction

Hiding complex implementation details and exposing only relevant data and functions



Polymorphism

Using methods with the same name but different implementations based on object type

These four principles are key pillars of object oriented programming that facilitate code reuse, simplicity, and flexibility.

Inheritance

Inheritance allows a class to inherit properties and behaviors (attributes and methods) from another class.

- 1 | What do most animals do?
Eat and sleep
- 2 | What is something dogs do?
Bark

Encapsulation

Think of encapsulation as a capsule or a container that holds something inside. In programming, encapsulation is like putting the important parts of an object inside a protective shell and only allowing access to them in controlled ways.

Money inside the piggy bank





Abstraction is like using a device without knowing how it works inside. It hides the complex details and exposes only the essential parts you need to interact with. It's like driving a car or using a remote control; you know what the controls do, but you don't need to know how they do it. Abstraction makes code easier to use and understand by focusing on what it does, not how it does it.

```
class Television:
    def turn_on(self):
        # Complex code to turn on the TV
        print("TV is on")

    def change_channel(self, channel):
        # Complex code to change the channel
        print(f"Changed to channel {channel}")

    def turn_off(self):
        # Complex code to turn off the TV
        print("TV is off")
```

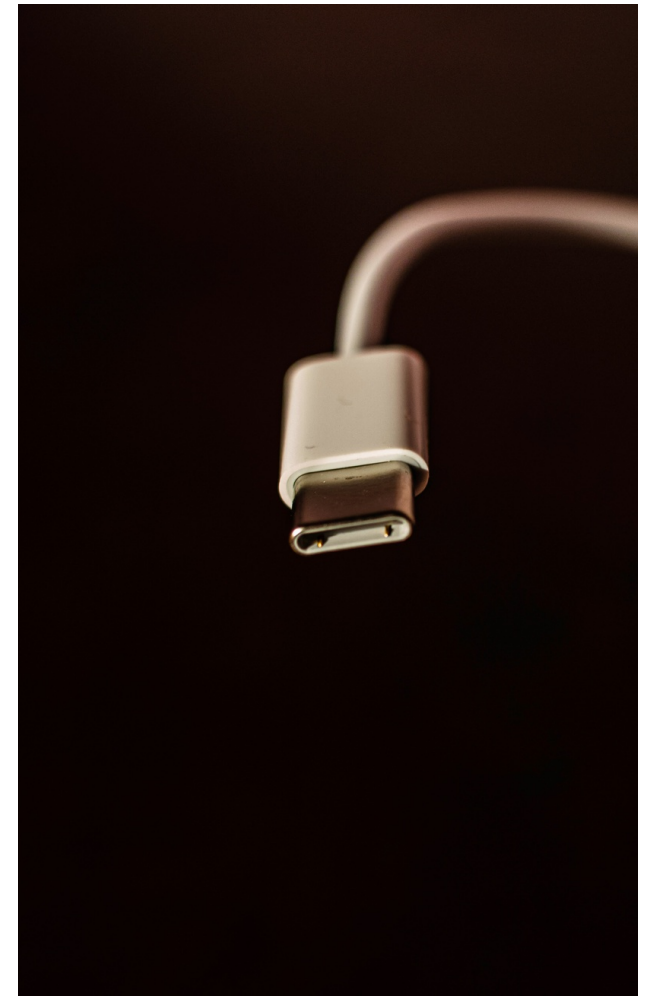
Abstraction

Polymorphism

Polymorphism is a term that means "many shapes."

In programming, it refers to the ability of different objects to be treated as instances of the same class or interface. It allows objects of different types to be treated uniformly.

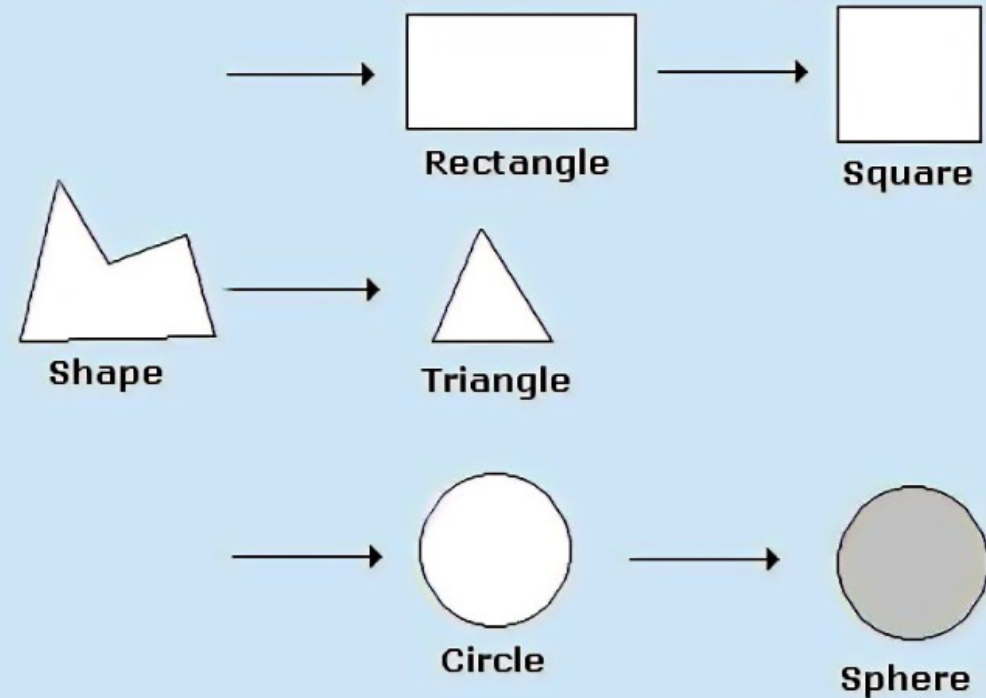
Polymorphism is like having a universal tool that can work with different things similarly. It allows objects of different types to be treated as the same type, enabling more flexible and reusable code.



Summary

Object Oriented Programming aims to organize complex programs into simple, reusable components called classes and objects. The main benefits of using OOP are increased code reusability, easier maintenance, extensibility through inheritance, and abstraction to hide unnecessary details. The four key principles of OOP - inheritance, encapsulation, abstraction, and polymorphism - make it an effective paradigm by enabling objects to share behaviors, expose only relevant details, and inherit functionality.

C++ Object Oriented Programming



Vic Broquard
Broquard eBooks